Magpie — Sprint 4 Technical Spec (Phase 2 finale: Category Schemas + Catalogue-Assisted Research)

Audience: Codex + Regan. **Builds on** Sprint 0–3. **Status: planning / review — implement only after this spec is reviewed and approved.**

> Build only the locked Sprint 4 scope (D-39…D-46). Listing generation, eBay, OCR/AI, realized-profit dashboards, attribute filtering, and any catalogue-data ingestion are **out of scope**. Sprint 4 **finishes Phase 2**.

Inherited from Sprint 0–3 (not restated): `TimeStampedUUIDModel`; `apps/` + `integrations/` ports; DRF defaults; **Decimal** for money/weight; the **real-PostgreSQL CI gate** (D-22). The schema registry (`apps/catalog/profiles.py`), `PreciousMetalsSchema`, the valuation strategies (incl. `commodity_manual` / `commodity_live` + their weight/fineness requirement at report create), `research/links.py`, and the cockpit all exist. **Sprint 4 adds no new Django app and no new models — no new migrations are expected.**

---

0. Locked decisions this spec implements

- **D-39** Sprint 4 = `StampSchema` + `CoinSchema` + `PhoneSchema` + category-aware research links/checklists + descriptor-driven forms. Tools/collectibles/bulk-lots stay permissive. Finishes Phase 2.
- **D-40** All schemas follow the **D-36 partial-capture pattern**: nothing required to save; reject invalid-when-present; unknown keys rejected; existing rows grandfathered (validated on next write); no point-of-use requirement in Sprint 4.
- **D-41** Locked field sets (§2.2–2.4): `catalogue_refs` = list of `{system, number}` (no flat per-system fields); **IMEI accepted as any string this sprint** (Luhn/warning validation deferred); **no `condition_grade`** (core `InventoryItem.condition` is the single source of truth).
- **D-42 Bullion Option A:** optional `weight_g` / `fineness` on `CoinSchema`; required only at commodity-valuation create (existing Sprint 3 enforcement — no new code); no duplication of the gold schema beyond those two fields.

- **D-43** Category-aware research links + **URL-less checklist entries**: per-profile builder registry; attribute-built queries with **fail-soft** fallback to title/SKU/generic; payload entries `{type: link|checklist, label, url (nullable), note, source}`; **honest source labels**; exact URL templates **verified at build time** (not permanent facts). Renniks = checklist; Stanley Gibbons = dealer-asking shop search.
- **D-44 Schema descriptors:** each registered schema exposes `fields()`; served via `GET /api/categories/{id}/schema/`; the frontend renders category forms from descriptors. Descriptor kept simple — **no dynamic-form framework**.
- **D-45 No catalogue-value ingestion / warehousing / fetching.** Values enter as `kind=catalogue Comparable`s (Sprint 2 machinery). IMEI/serial must **never auto-flow into future listing text** (recorded constraint for Phase 3).
- **D-46** Out-of-scope list (see §9).

---

1. What Sprint 4 adds / changes

File	Change
`apps/catalog/profiles.py`	Add a tiny `FieldSpec` helper + `fields()` to the schema protocol; add `StampSchema`, `CoinSchema`, `PhoneSchema`; **retrofit `PreciousMetalsSchema` with `fields()`** (validation rules unchanged).
`apps/catalog/views.py` + `apps/api`	`GET /api/categories/{id}/schema/`.
`apps/research/links.py`	Per-profile link-builder registry; checklist entry support; extended entry payload.
`apps/core/management/commands/seed.py`	Realistic stamp/coin/phone sample attributes (incl. one bullion sovereign).
`frontend/`	`SchemaFieldsForm` (descriptor-driven renderer) replacing the bespoke gold section; `ResearchLinks` renders checklist rows + source labels.

---

2. Backend

2.1 `FieldSpec` + descriptor contract (`apps/catalog/profiles.py`)

Define each schema's fields **once**; derive both `validate()` and `fields()` from the same definitions (single source of truth — this is what D-44 buys).

```
@dataclass(frozen=True)
class FieldSpec:
    name: str
    label: str
    type: str                       # "str" | "int" | "decimal" | "choice" | "obje
    choices: tuple = ()             # for type="choice"
    min: Decimal | int | None = None
    max: Decimal | int | None = None
    help_text: str = ""
    item_shape: dict | None = None # for object / list[object]: {"system": {...},

class AttributeSchema(Protocol):
    profile_key: str
    def validate(self, attributes: dict) -> dict: ...
    def fields(self) -> list[dict]: ...   # serialized FieldSpecs; PermissiveSche
```

Validation per the D-36 pattern: every field optional to save; when present — type-coerce, range-check, choice-check; reject unknown keys; raise `ValueError` with a per-field message. `required` is always false in descriptors this sprint.

2.2 `StampSchema` (`profile_key = "stamps"`)

Field	Rule when present
`country`	str
`year`	int, 1840 – current_year+1
`denomination`	str ("2d", "45c")
`face_value_currency`	str (short, free — predecimal units aren't ISO)
`topic_theme`	str ("birds", "ships")
`catalogue_refs`	list of `{system, number}` ; system ∈ {SG, Scott, Michel, Yvert, ACSC, other}

`mint_used`	**choice:** `mint_never_hinged / mint_hinged / used / cto / unknown`
`perforation` / `watermark` / `variety` / `colour` / `notes`	**str**

2.3 `CoinSchema` (`profile_key = "coins"`)

Field	Rule when present
`country` / `denomination` / `mint_mark` / `ruler_or_reign`	**str**
`year`	**int, –1000 – current_year+1 (ancients exist)**
`composition`	**str ("92.5% silver")**
`grade`	**str — free text; UI *suggests* VG/F/VF/EF/aUNC/UNC + slab grades; no grading engine**
`catalogue_refs`	**list of** `{system, number}` **; system ∈ {KM, Renniks, other}**
`mintage`	**int ≥ 0**
`cert`	**object** `{grader ∈ {PCGS, NGC, other}, cert_no: str}`
`obverse_type` / `reverse_type`	**str**
`weight_g`	**Decimal > 0** ***(bullion — D-42)***
`fineness`	**Decimal in (0, 1]** ***(bullion — D-42)***
`notes`	**str**

Bullion note: a coin-category item with `weight_g` + `fineness` can be valued via `commodity_manual` / `commodity_live` **with zero new code** — the Sprint 3 report-create validation (weight + fineness/karat required) is category-agnostic. One test proves it (§7).

2.4 `PhoneSchema` (`profile_key = "phones"`)

Field	

	Rule when present
`brand` / `model` / `colour` / `carrier`	**str**
`storage_gb` / `ram_gb`	**int > 0**
`imei`	**str — any string accepted this sprint** (D-41); no Luhn
`serial_no`	**str**
`network_status`	**choice:** `unlocked / carrier_locked / unknown`
`battery_health_pct`	**int 0 – 100**
`faults`	**str (buyer-material defects — distinct from internal** `notes` **)**
`accessories` / `notes`	**str**

(No *`condition_grade`* ***— core*** *`InventoryItem.condition`* ***is the single source of truth, D-41.)***

2.5 Registration

```
register(StampSchema()); register(CoinSchema()); register(PhoneSchema())
```

Seeded categories get `profile_key` **set: Stamps →** `"stamps"` **, Coins →** `"coins"` **, Phones & Electronics →** `"phones"` **(Gold already** `"gold"` **). Tools / Collectibles / Bulk Lots / Unknown keep** `""` **(permissive).**

2.6 Research links (`apps/research/links.py`)

```
# Entry shape (extends Sprint 2's {label, url} additively):
# {"type": "link"|"checklist", "label": str, "url": str|None, "note": str, "sourc

LINK_BUILDERS: dict[str, Callable] = {}   # profile_key -> builder(item, attrs) -

def research_links(item) -> list[dict]:
    entries = generic_links(item)                      # Sprint 2 set, now type="
    builder = LINK_BUILDERS.get(item.category.profile_key if item.category else "
    if builder:
        entries += builder(item, item.attributes or {})
```

```
    return entries
```

Builders (queries from attributes; **fail soft** to title/SKU; templates below are *indicative — verify at build time*, D-43):

- **stamps:** Colnect stamps search (q = country + year + denomination [+ topic]); **Stanley Gibbons shop search —** `source: "dealer asking (shop search) — not catalogue value"`.
- **coins:** Numista search (q = country + year + denomination); Colnect coins; **PCGS** `pcgs.com/cert/{cert_no}` **/ NGC** `ngccoin.com/certlookup/{cert_no}/` **only when** `cert` **present** (source: "cert verification"); **Renniks → checklist entry**: `{type: "checklist", url: None, label: "Check Renniks AU Coin & Banknote Values (latest ed.)", note: "Print catalogue — no public searchable DB. Enter values as kind=catalogue comps.", source: "manual checklist"}`.
- **phones:** GSMArena search (q = brand + model, source: "spec database"); brand+model+storage variants of the existing eBay sold/active queries.

2.7 Schema endpoint (`apps/catalog/views.py`)

`GET /api/categories/{id}/schema/` → `{"profile_key": str, "fields": [...]}`.
Permissive categories → `{"profile_key": "", "fields": []}`.

---

3. API

Method	Path	Purpose
GET	`/api/categories/{id}/schema/`	Field descriptors for the category's schema (empty for permissive).
GET	`/api/items/{id}/research-links/` *(existing path)*	Extended payload: generic + category entries, `type` / `note` / `source` added, checklist entries have `url: null`.

Everything else is unchanged. Attribute validation continues to run through the existing item create/update path (model `clean()` → schema registry).

---

4. Frontend

- **`SchemaFieldsForm`** — fetches the category schema descriptor and renders inputs by type: `str` → text, `int` / `decimal` → number, `choice` → select, `object` → inline fieldset, `list[object]` → repeatable rows (add/remove — used by `catalogue_refs`). Used in **AddItem** and **ItemDetail/ItemForm** whenever the descriptor is non-empty. **Replaces the bespoke gold field section** — gold renders via its retrofitted descriptor with visual parity (acceptance criterion).
- **`ResearchLinks`** — link entries stay buttons (new-tab); **checklist entries render as non-clickable checklist rows** with the note; every entry shows its `source` tag (public search / dealer asking / cert verification / spec database / manual checklist).
- **No** attribute filtering, completeness indicators, dashboards, or per-category analytics (D-46).

---

5. Config

No new env vars. No new migrations expected (assert in CI that `makemigrations --check` is clean apart from any `profile_key` data seeding done in `seed.py`, which is data, not schema).

---

6. Acceptance criteria

1. **Postgres CI gate green** (D-22). No new migrations; `makemigrations --check` clean.
2. `StampSchema`, `CoinSchema`, `PhoneSchema` registered; seeded categories mapped to their `profile_key`s; **partial capture saves** for all three; invalid values rejected **when present** (year bounds, `battery_health_pct` 0–100, `fineness` ∈ (0,1], `mintage` ≥ 0, bad choices, malformed `catalogue_refs` / `cert`); unknown keys rejected; pre-existing rows still load/save (grandfathered — validated on next write).
3. `catalogue_refs` round-trips as a `{system, number}` list; `cert` as `{grader, cert_no}`.
4. **Bullion:** a coin-category item with `weight_g` + `fineness` produces a `commodity_manual` and (fake-adapter) `commodity_live` report; missing weight/fineness → 422 — proving strategy/category decoupling with **no new valuation code**.
5. `GET /api/categories/{id}/schema/` returns correct descriptors for stamps/coins/phones/**gold**; `[]` for permissive categories.

6. Item forms render **from descriptors** for all four schema'd categories; the hand-coded gold section is removed; gold has visual parity.
7. Research links: category builders emit attribute-built queries with title/SKU fail-soft; **PCGS/NGC deep-links only when `cert` present**; **Renniks appears as a checklist entry**; SG labelled as dealer asking; every entry carries an honest `source`.
8. **IMEI accepts any string; no `condition_grade` anywhere.** No forbidden features (§9).
9. Backend + frontend tests pass; `backup` /restore unaffected (no new tables).

---

7. Test plan

Backend (`pytest` + `pytest-django`):

- *Schemas:* per category — empty attrs save; each invalid-when-present case rejected with a clear message; unknown key rejected; valid full set round-trips; `catalogue_refs` / `cert` shape validation.
- *Grandfathering:* an item saved pre-Sprint-4 with permissive attrs loads; next write validates.
- *Descriptors:* endpoint returns the four field sets; permissive → empty; descriptor `type` s match validation behaviour (spot-check int/choice/list[object]).
- *Links:* per-profile builders produce expected entries; cert deep-link present/absent with/without `cert` ; Renniks checklist present for coins; fail-soft with empty attrs (falls back to title/SKU); payload shape `{type, label, url, note, source}`.
- *Bullion:* coin item + weight/fineness → `commodity_manual` report computes; `commodity_live` with `FakeMetalsPriceAdapter` computes; missing weight → 422.

Frontend (Vitest + RTL — light): `SchemaFieldsForm` renders text/number/select/repeatable-list from a fixture descriptor; gold parity smoke; checklist rows render non-clickable with note + source; links row shows source tags.

CI: Postgres gate; fake metals adapter only; no external calls of any kind (link builders are pure URL templating — assert no HTTP in tests).

---

8. Seed data (extend `seed.py` , idempotent)

- **Stamp:** Australia 1932 "Sydney Harbour Bridge" 2d, `{country: "Australia", year: 1932, denomination: "2d", face_value_currency: "AUD (pre-decimal d)", catalogue_refs: [{system: "SG", number: "144"}], mint_used: "used", topic_theme: "bridges"}` ***(labelled seed/example — verify the SG number is plausible, it's illustrative not authoritative)***.
- **Coins:** one numismatic (1937 Australian Crown, `{ruler_or_reign: "George VI", grade: "VF", catalogue_refs: [{system: "KM", number: "34"}], mintage: 1008000}`) **and one bullion sovereign (** `{weight_g: "7.988", fineness: "0.9167", composition: "22ct gold", ruler_or_reign: "George V"}` **) — the sovereign gets a** `commodity_manual` **report to exercise D-42.**
- **Phone:** `{brand: "Samsung", model: "Galaxy S21", storage_gb: 128, ram_gb: 8, network_status: "unlocked", battery_health_pct: 87, faults: "light scratch rear glass"}` .
- **All clearly labelled seed/example data.**

---

9. Out of scope for Sprint 4 (restated — D-46)

Tools / collectibles / bulk-lot schemas · JSONB attribute filtering or attribute-search UI · schema-completeness indicators · per-category dashboards/analytics · promoted columns · forced backfill of existing rows · catalogue-value ingestion, scraping, or fetching of catalogue pages · eBay Browse/write · listing generation · OCR/AI · `SaleRecord` **/realized-profit dashboards · IMEI Luhn/warning validation (deferred) · automated trading/buying.**

Hand-off notes for the implementing agent

- **Define fields once (** `FieldSpec` **) and derive both** `validate()` **and** `fields()` **— do not maintain two parallel field lists.**
- **Retrofit gold** with `fields()` but do **not** change its validation behaviour; the gold form must look/work the same through the new renderer.
- **Verify every external URL template at build time** (Numista/Colnect/GSMArena/PCGS/NGC/SG); if one isn't templatable, demote it to a checklist entry rather than shipping a broken link. Link builders are **pure URL templating — no HTTP anywhere**.

- The links payload change is **additive** — don't break the Sprint 2 generic entries.
- No new models, no new migrations. If you find yourself writing a migration (beyond none), stop and re-read this spec.
- When ambiguous, prefer the smaller implementation that satisfies §6.